

Lab4: Sakiko's Delivery System 实验报告

学生信息

- 姓名：黄雯佩
- 学号：PB24111630

1. 实验目的

本实验旨在通过 LC-3 汇编语言实现一个递归程序。通过模拟送货员 Sakiko 在网格化城市中的路径规划问题，练习以下核心概念：

递归算法实现：根据给定的数学递推公式计算路径总数 $\text{Routes}(i, j)$ 。

堆栈管理：利用 R6 寄存器维护程序栈，实现递归过程中的参数保护与环境恢复。

算术运算：在指令受限的 LC-3 环境中实现加法（Steps 计算）和乘法（Recommendation 公式计算）。

2. 实验过程

2.1 算法实现步骤

1. **输入加载：**从内存地址 x3100(N) 和 x3101(M) 加载坐标值。
2. **Steps 计算：**直接执行 $N+M$ 。
3. **递归求解 Routes：**
 - Base Case：**若 $i=0$ 或 $j=0$ ，结果为 1。
 - Recursive Step：**返回 $\text{Routes}(i-1, j) + \text{Routes}(i, j-1)$ 的和。
4. **公式整合：**利用多次累加实现 $\text{Routes} \times 5$ ，随后减去 Steps 得到最终推荐值。

2.2 难点-栈的分配与管理

1. 栈帧结构设计

为了支持深层递归并防止寄存器数据被后续调用覆盖，每个 `ROUTES_RECURSIVE` 函数实例都会在内存中分配一个固定结构的栈帧：

R6 + 2: 存放返回地址 (R7)，确保子程序执行完毕后能准确跳回调用点。

R6 + 1: 存放当前递归层级的参数 i (即寄存器 R0 的快照)。

R6 + 0: 存放当前递归层级的参数 j (即寄存器 R1 的快照)。

2. 动态分配与释放流程

入栈分配：子程序入口处使用 `ADD R6, R6, #-3` 开辟空间，并利用 `STR` 指令将环境信息存入栈中。

中间值保护：由于 $\text{Routes}(i, j) = \text{Routes}(i-1, j) + \text{Routes}(i, j-1)$ ，在完成第一个分支计算后，程序执行 `ADD R6, R6, #-1` 将第一个结果压栈保存，防止其在第二个分支递归时被 R0 的返回值覆盖。

环境恢复：

在执行第二个递归分支前，利用 `LDR R0, R6, #2` 重新从栈中读取原始参数 i 。

在返回前，利用 `LDR R7, R6, #2` 恢复返回地址，并通过 `ADD R6, R6, #3` 彻底释放该层级占用的栈空间。

3. 实验结果

通过C语言验证程序对 $0 \leq N, M \leq 5$ 范围内的所有解进行了测算。以下为部分典型测试用例：

C语言代码

```
lab4.c > main()
1  #include <stdio.h>
2
3  int calculate_routes(int i, int j) {
4      // 基准情况 (Base Case) [cite: 21]
5      if (i == 0 || j == 0) {
6          return 1;
7      }
8      return calculate_routes(i - 1, j) + calculate_routes(i, j - 1);
9  }
10
11 int get_recommendation(int n, int m) {
12     int steps = n + m; // [cite: 16]
13     int routes = calculate_routes(n, m); // [cite: 17]
14
15     return (routes * 5) - steps;
16 }
17
18 int main() {
19     for (int n = 0; n <= 5; n++) {
20         for (int m = 0; m <= 5; m++) {
21             int res = get_recommendation(n, m);
22             printf("(%d,%d):%4d", n,m,res);
23         }
24         printf("\n");
25     }
26     return 0;
27 }
```

C语言结果

```
(3,4): 168 ...
PS E:\CoursesData\ICS\labs\lab4> & 'c:\Users\18206\.vscode\extensions\ms-vscode.cpptools-1.29.3-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-okwrx3r.ibu' '--stdout=Microsoft-MIEngine-Out-mzq24bev.02b' '--stderr=Microsoft-MIEngine-Error-22df54b1.yzu' '--pid=Microsoft-MIEngine-Pid-ncmivgfa.m0s' '--dbgExe=c:\x86_64-8.1.0-release-win32-seh-rt_v6-rev0\mingw64\bin\gdb.exe' '--interpreter=mi'
(0,0): 5 (0,1): 4 (0,2): 3 (0,3): 2 (0,4): 1 (0,5): 0
(1,0): 4 (1,1): 8 (1,2): 12 (1,3): 16 (1,4): 20 (1,5): 24
(2,0): 3 (2,1): 12 (2,2): 26 (2,3): 45 (2,4): 69 (2,5): 98
(3,0): 2 (3,1): 16 (3,2): 45 (3,3): 94 (3,4): 168 (3,5): 272
(4,0): 1 (4,1): 20 (4,2): 69 (4,3): 168 (4,4): 342 (4,5): 621
(5,0): 0 (5,1): 24 (5,2): 98 (5,3): 272 (5,4): 621 (5,5):1250
PS E:\CoursesData\ICS\labs\lab4>
```

结果截图 (0,0):5

Memory			
!	▶ x3100	x0000	0
!	▶ x3101	x0000	0
!	▶ x3102	x0000	0
!	▶ x3103	x0000	0
!	▶ x3104	x0000	0
!	▶ x3105	x0000	0

memory			
!	▶ x3200	x0005	5
!	▶ x3201	x0000	0
!	▶ x3202	x0000	0
!	▶ x3203	x0000	0

(3,0):2

Memory			
!	▶ x3100	x0003	3
!	▶ x3101	x0000	0
!	▶ x3102	x0000	0
!	▶ x3103	x0000	0
!	▶ x3104	x0000	0

Memory			
!	▶ x3200	x0002	2
!	▶ x3201	x0000	0
!	▶ x3202	x0000	0
!	▶ x3203	x0000	0
!	▶ x3204	x0000	0

(4,2):69

Memory			
!	▶ x3100	x0004	4
!	▶ x3101	x0002	2
!	▶ x3102	x0000	0
!	▶ x3103	x0000	0
!	▶ x3104	x0000	0
!	▶ x3105	x0000	0

Memory			
!	▶ x3200	x0045	69
!	▶ x3201	x0000	0
!	▶ x3202	x0000	0
!	▶ x3203	x0000	0
!	▶ x3204	x0000	0

(5,5):1250

Memory			
!	▶ x3100	x0005	5
!	▶ x3101	x0005	5
!	▶ x3102	x0000	0
!	▶ x3103	x0000	0
!	▶ x3104	x0000	0

Memory			
!	▶ x3200	x04E2	1250
!	▶ x3201	x0000	0
!	▶ x3202	x0000	0
!	▶ x3203	x0000	0
!	▶ x3204	x0000	0

4. 讨论

4.1 为什么递归方法效率较低？

重复计算：该递归算法存在大量重复子问题。例如计算 (2,2) 时，会分别计算 (1,2) 和 (2,1)，而这两者都会再次独立触发对 (1,1)的计算。随着N, M增大，计算量呈指数级增长。

堆栈开销：每次函数调用都需要执行多次压栈（Push）和出栈（Pop）操作，涉及大量的内存访问（LDR/STR），相比简单的循环指令开销巨大。

4.2 如何提高程序效率？

备忘录法：开辟一块内存区域存储已计算出的 Routes(i, j)结果。在递归开始前先查表，若已有结果则直接返回，避免重复计算。

迭代法：使用动态规划的思想，从(0,0)开始逐行或逐列填充表格，仅需O(N*M)的时间复杂度且无需维护复杂的函数栈。